M.Tech | Advanced Software Engineering

Microservices Based Development Using DevOps



SUBMITTED BY

Balla Bharath Venkat (2610603)
Rohit Bhure (2610611)

PROGRAM

Master of Technology (M.Tech)

DEPARTMENT

Information Architecture & Software Engineering

SUBJECT

Advanced Software Engineering

GUIDE

Dr. Ugrasen Suman

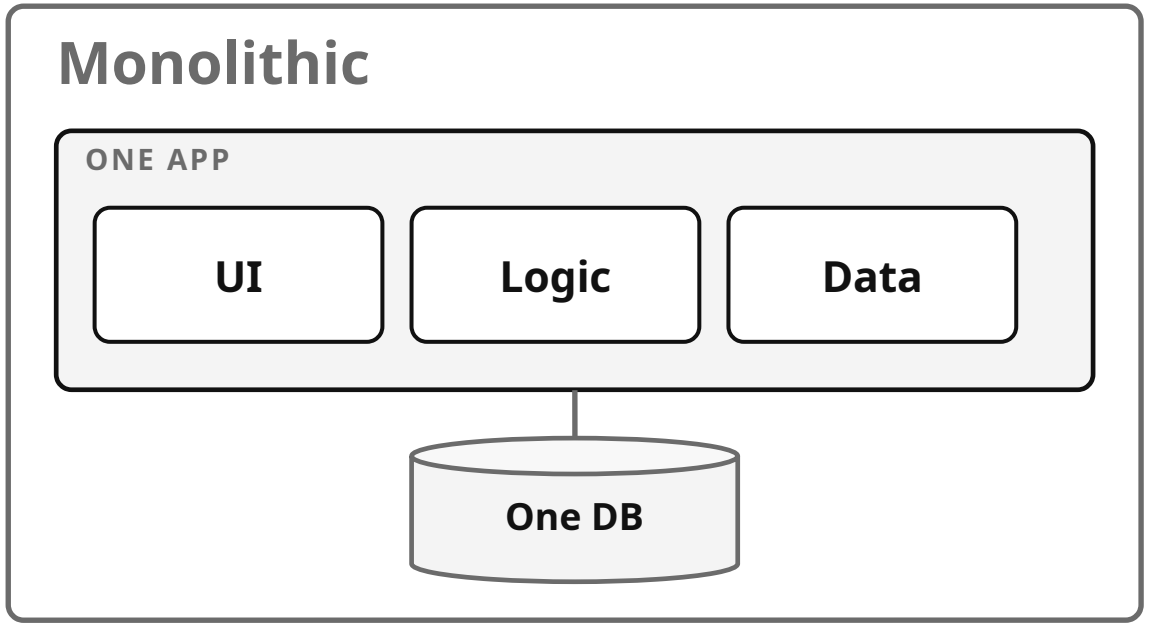
INSTITUTION

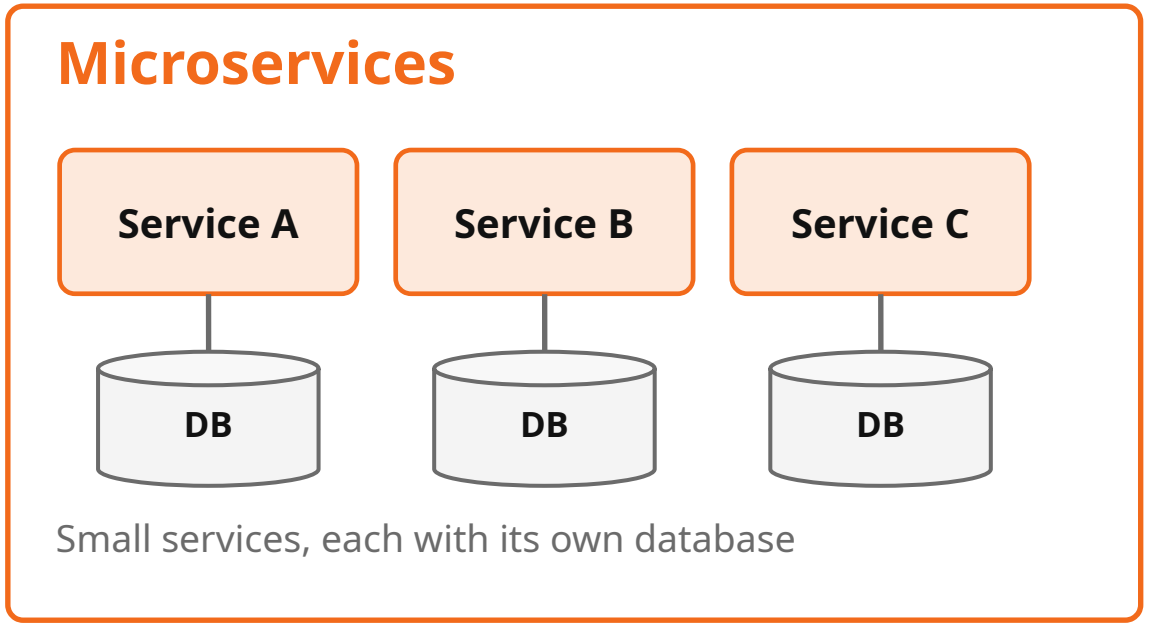
School of CS & IT, DAVV, Indore

SESSION

2026-2027

Monolithic vs Microservices: The Idea





- One change = redeploy everything
- Cannot scale just one part
- Tightly coupled, one tech stack

- Deploy each service on its own
- Scale only what is needed
- Each team picks its own tech

Microservices: Key Characteristics

Microservices = an app built as many small services. Each does one business job and is deployed on its own. (Balalaie et al., 2016)

One Job per Service

e.g., User Login, Orders, Payments

Independent Deployment

Release one service without touching others

Own Database

No shared schema between services

Any Technology

Java, Node.js, Python, Go — team's choice

Fault Isolation

One service fails, the system keeps running

Light Communication

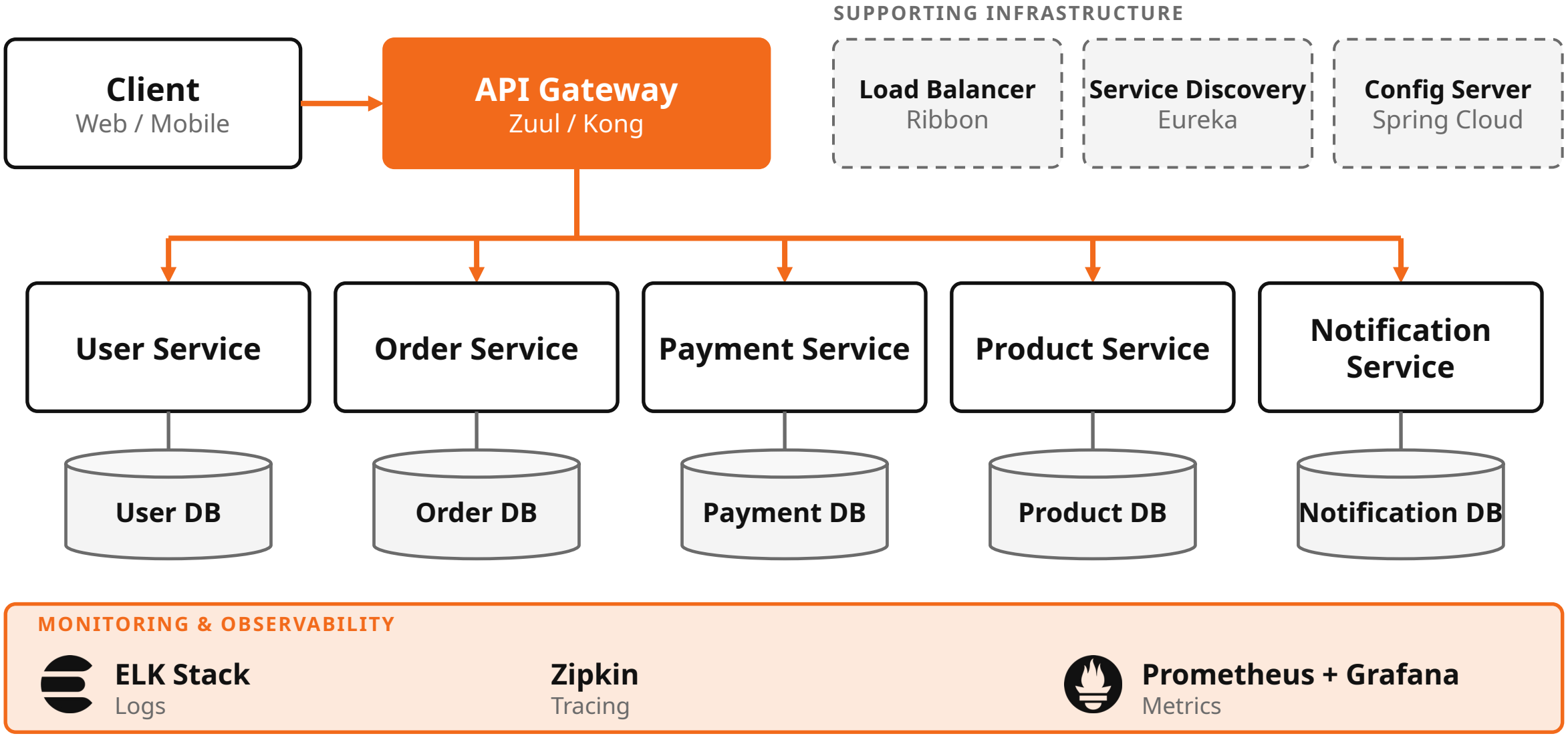
REST APIs or message queues (Kafka, RabbitMQ)

Made possible by: Docker • Kubernetes • CI/CD • Cloud

Monolithic vs. Microservices

	Monolithic	Microservices
Deployment	One big package	Many small packages
Scaling	Scale the whole app	Scale only one service
Technology	One stack	Best tool per service
Team	Large, separate teams	Small DevOps teams
Failure	One bug can crash all	Failure stays inside one service
Database	One shared database	Database per service

Microservices Architecture



DevOps and Why Microservices Need It

DevOps = Dev and Ops teams working together, with automation, to deliver software fast and continuously. (Bass, Weber & Zhu, 2015)

CI — Continuous Integration

Merge often, test automatically

CD — Continuous Delivery

Always ready to release

CM — Continuous Monitoring

Watch production in real time

IaC — Infrastructure as Code

Environments written as code

DevOps is a culture shift, not just a tool.

Why Microservices need DevOps

1. **Many services = many pipelines**

   One CI/CD pipeline per service

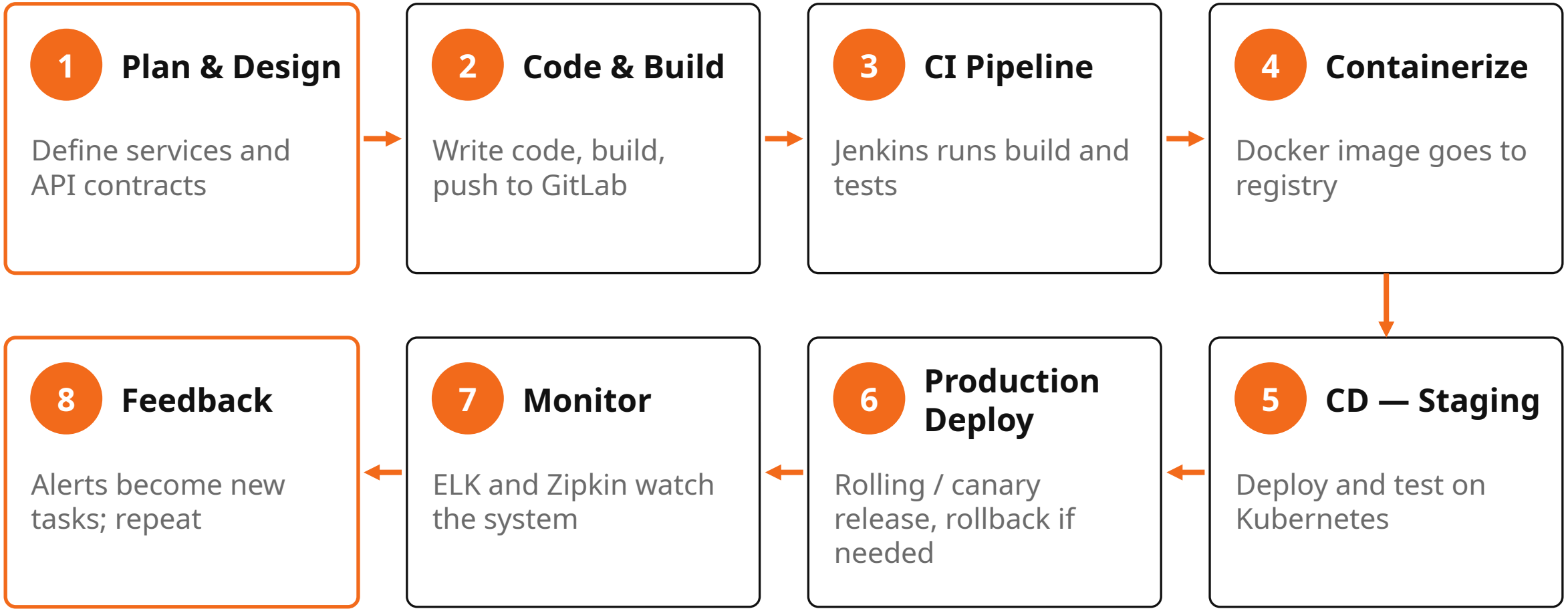
2. **Same environment everywhere**

   Docker containers

3. **Run and scale automatically**

   Kubernetes + monitoring

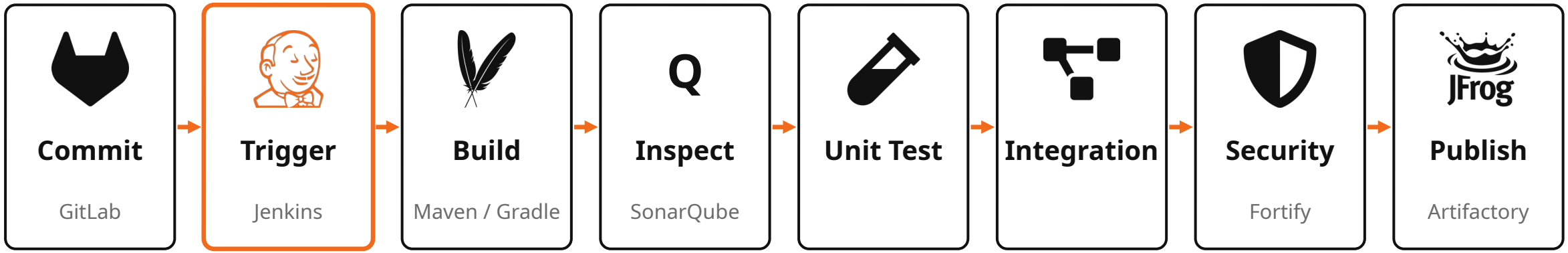
Microservices Development Lifecycle



Version Control (Git) and Continuous Integration

One Git repository per service. Code is merged to main only after CI passes. Backtory used self-hosted GitLab.

CI pipeline: every commit is built and tested automatically



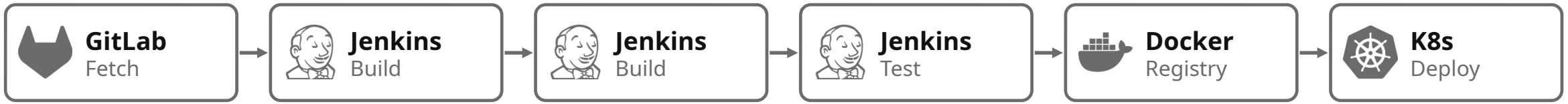
- Merge small changes often — avoids big integration conflicts
- Backtory: one CI job per microservice

CI/CD Pipeline: Old vs New

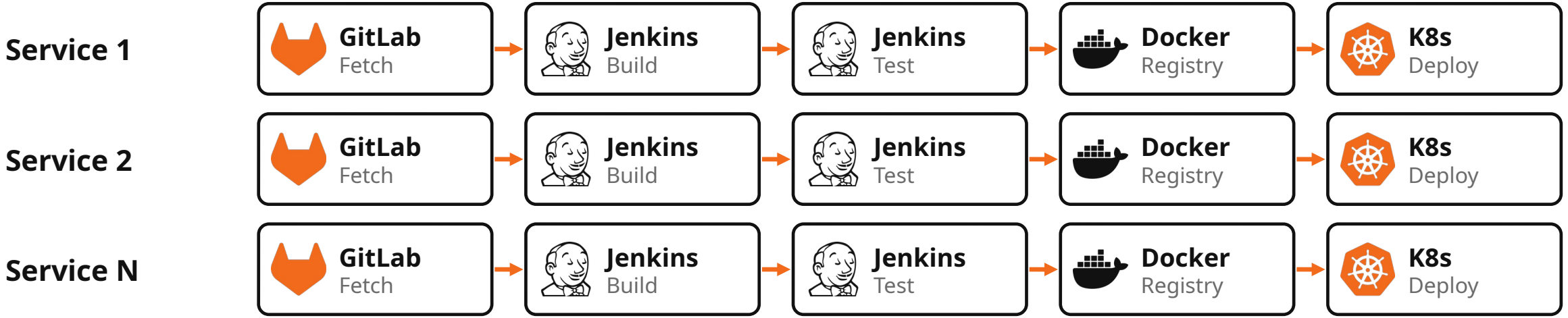
Continuous Delivery: every passing build is ready to release

Continuous Deployment: every passing build goes live automatically

(a) Old: one pipeline for the whole monolith



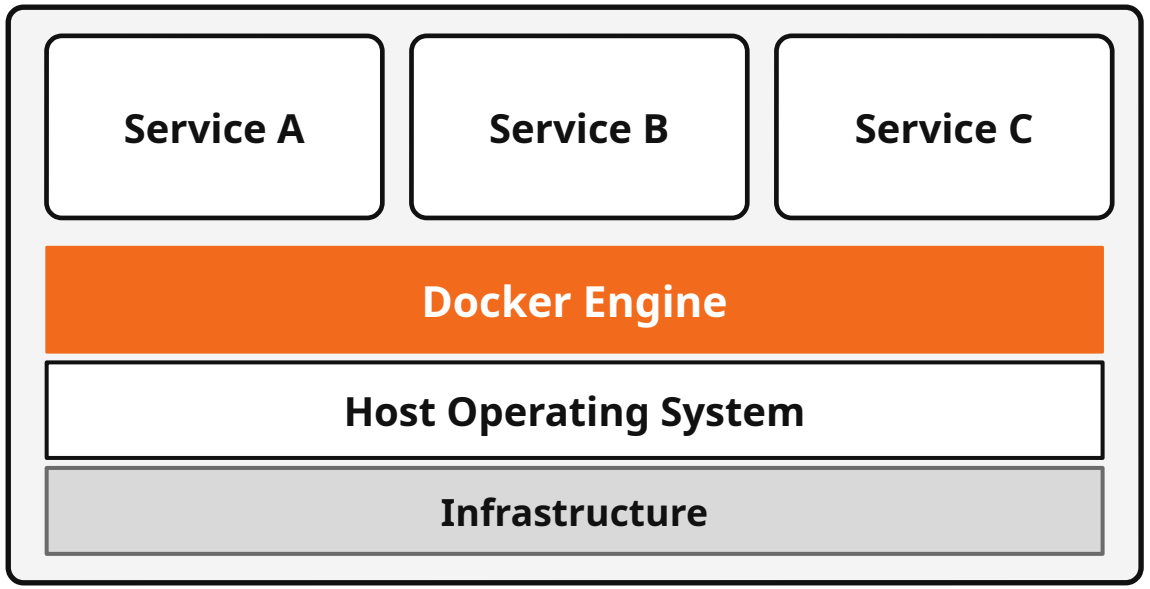
(b) New: separate pipeline for each microservice



Key difference: each service deploys independently — no other service is affected (Balalaie et al., 2016).

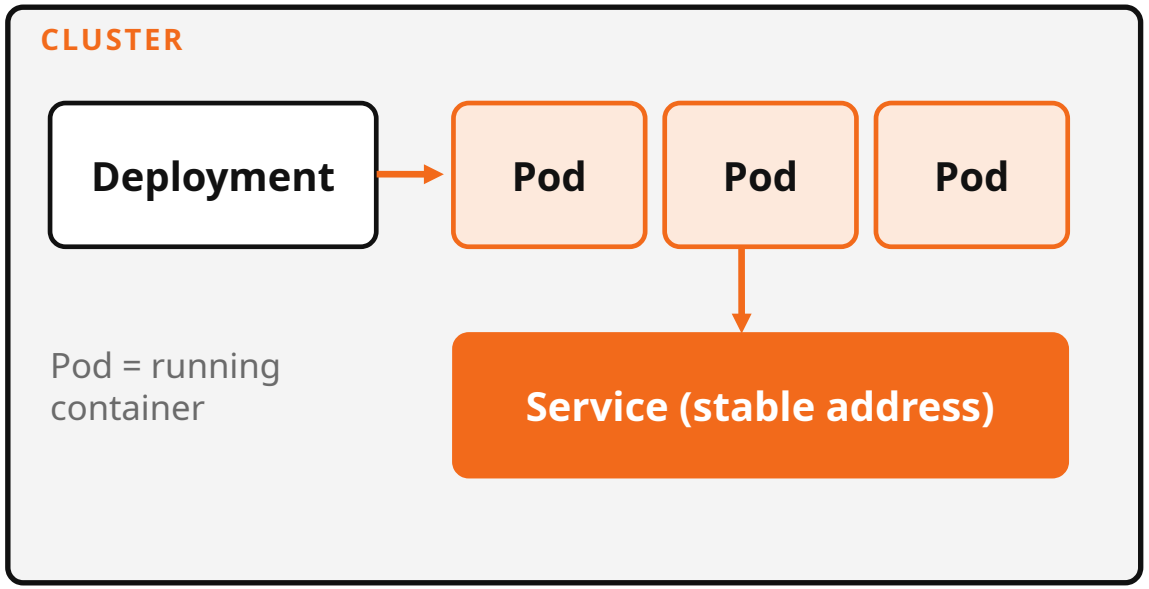
Docker and Kubernetes

Docker — package it



- Packs a service + its dependencies
- Same behaviour in dev, test, production
- Docker Compose: start all services with one file

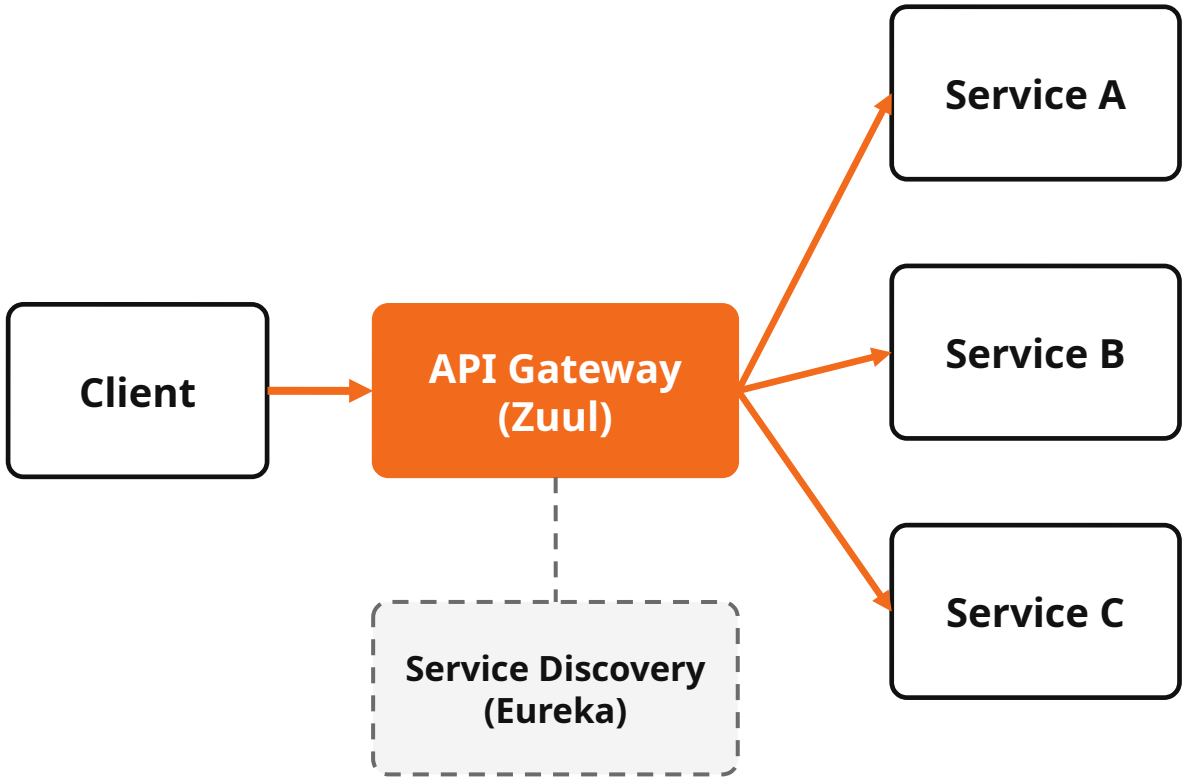
Kubernetes — run it



- Deploys containers automatically
- Scales up/down and heals failures
- Used with CoreOS in Backtory

Both papers: Docker + Kubernetes (Balalaie et al.; Xuan et al.)

API Gateway and Service Communication



API Gateway (Zuul)
Single entry point: routing, authentication

REST / gRPC
Direct calls between services (synchronous)

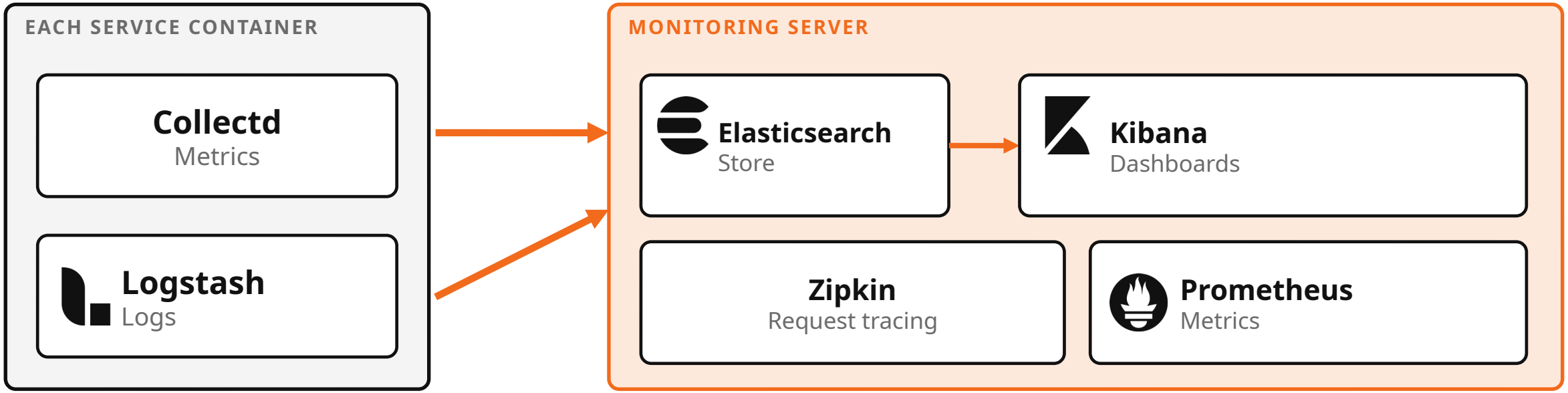
Message Queue
RabbitMQ / Kafka — async, loosely coupled

Service Discovery (Eureka)
Finds where each service is running

Circuit Breaker (Hystrix)
Fails fast so one failure does not spread

Container IPs change, so services use discovery instead of fixed addresses.

Monitoring, Logging and Tracing

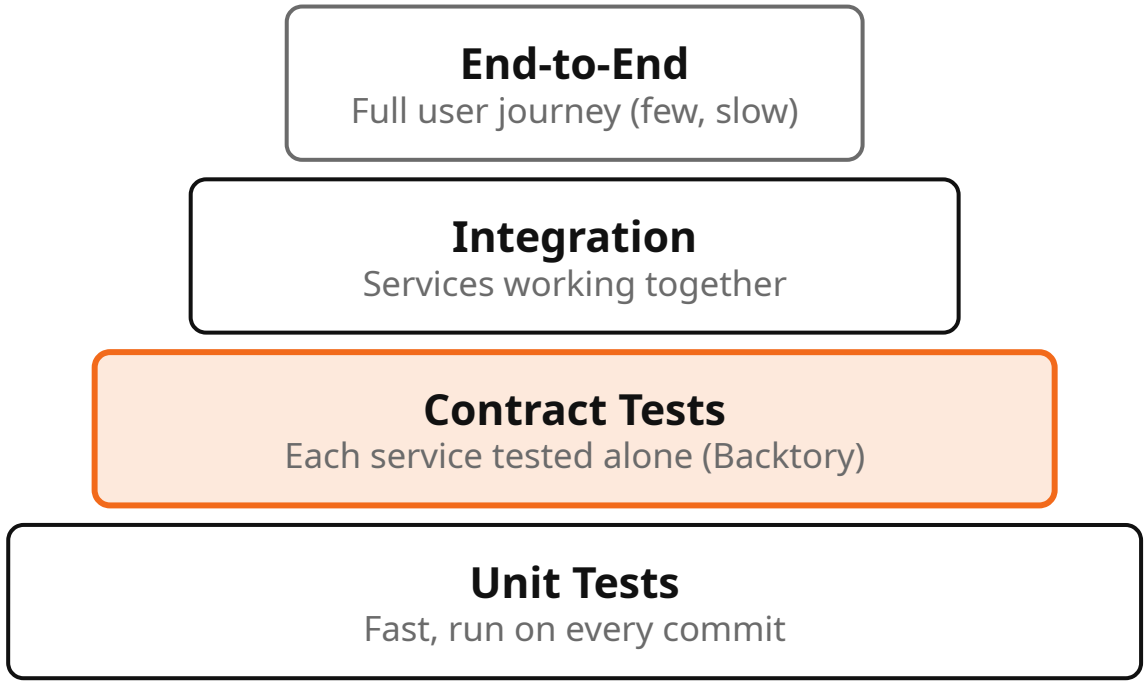


Per-service monitoring
Each service has its own metrics and logs

Distributed tracing
Zipkin shows which service is slow

Testing and Security

Testing



Tools (Xuan et al.)

- **Fortify**
- **Appscan**
- **JMeter**

Security (DevSecOps)

Security is checked at every stage, not only at the end.

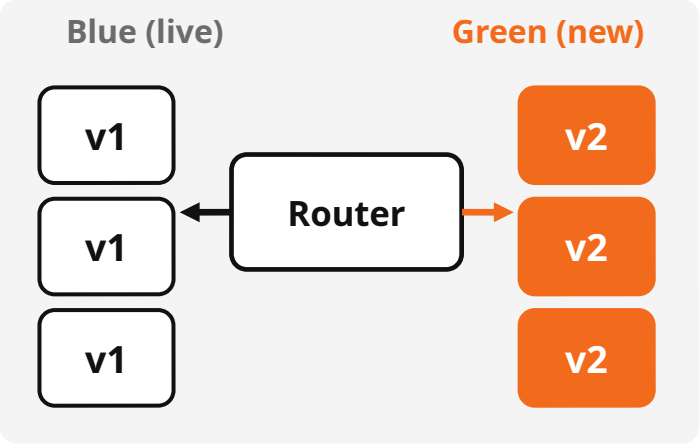
1. **Plan** Security rules in the design
2. **CI** Static code scan (Fortify)
3. **CD** Black-box scan (Appscan)
4. **Production** Monitor and patch continuously

Microservice risks

Many APIs • service-to-service trust • secrets • image scanning

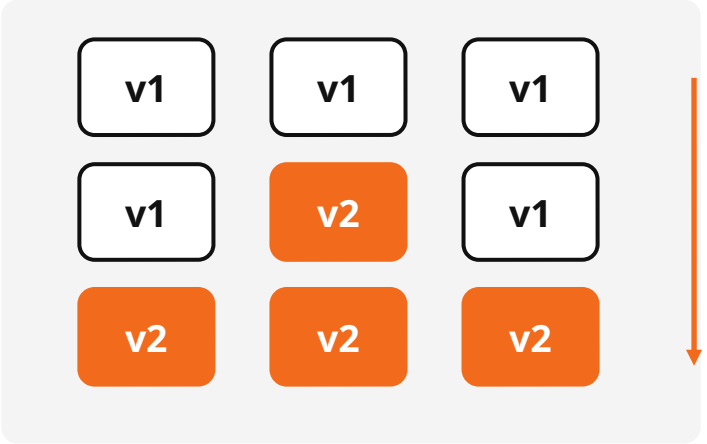
Deployment Strategies

Blue-Green



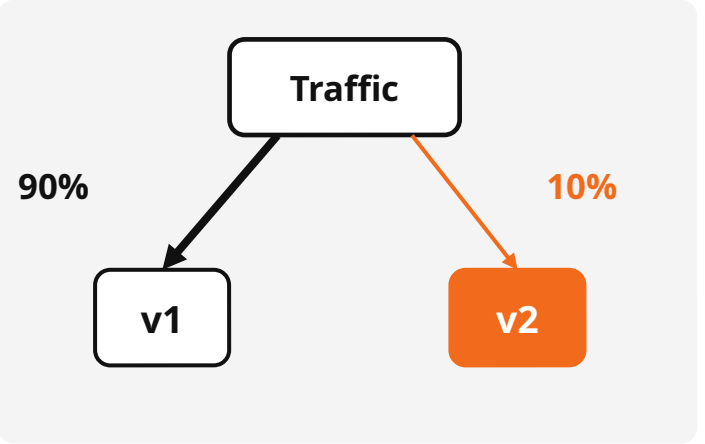
- Two identical environments
- Switch traffic in one step
- Instant rollback

Rolling



- Replace old pods gradually
- v1 and v2 run together
- Kubernetes default

Canary



- Small % of traffic to new version
- Watch errors, then increase
- Safest for risky changes

Challenges and Best Practices

Challenges	Best Practices
Distributed complexity Network failures, latency	→ **Design by business capability** Domain-Driven Design
Local dev setup Many services to run	→ **Docker Compose** Run everything locally
Service contracts Small API change can break others	→ **Consumer-driven contracts** Test each service alone
Skills gap Needs distributed-systems know-how	→ **Cross-functional teams** One team per service
Operations load Many deployments and dashboards	→ **Automate + monitor** CI/CD, ELK, Zipkin from day one

References

[1] Microservice Publishing Technology Based on DevOps Architecture
IEEE ITNEC(ISSN:2693-3128)

[2] Microservices Architecture Enables DevOps



THANK YOU

Questions & Discussion

Balla Bharath Venkat (2610603) | Rohit Bhure (2610611)
M.Tech — Information Architecture & Software Engineering
School of CS & IT, DAVV, Indore | Advanced Software Engineering
Guide: Dr. Ugrasen Suman | Session 2026-2027